

Project Context — Agentic AI + Data Engineering Platform

I want you to work with me as a long-term technical project partner on this project. Do not simply agree with my decisions. If you see a technically better, safer, simpler, or more production-oriented approach, challenge my decision and explain why. At the same time, do not redesign working components unnecessarily.

1. Project Goal

I am building a portfolio-level **Agentic AI + Data Engineering platform** that combines:

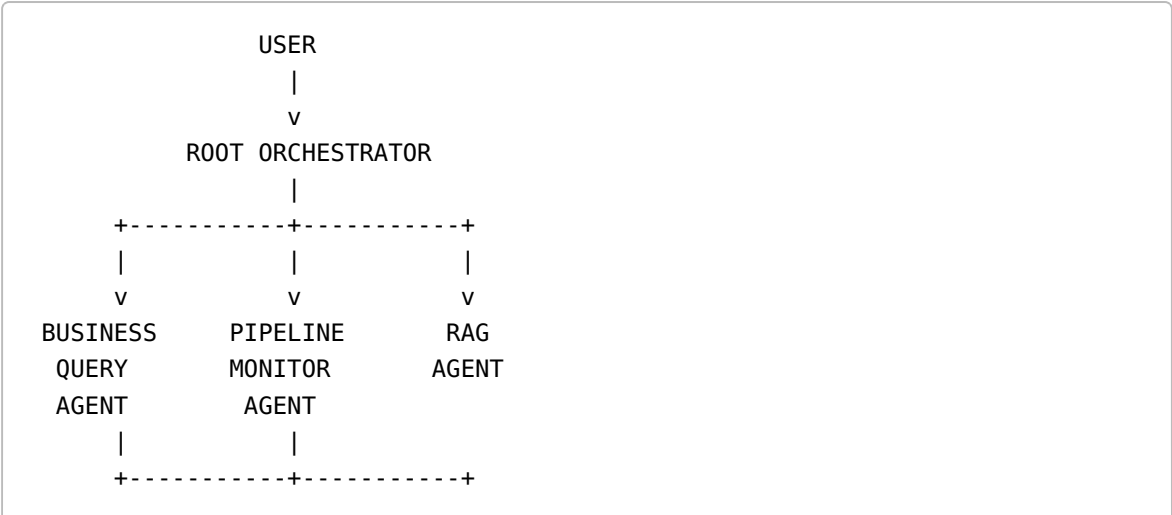
- Google ADK
- Agentic AI
- PySpark
- Apache Airflow
- MySQL
- MCP
- RAG
- LLM-based business analytics

The goal is not to build disconnected demos. I want one coherent system where the Data Engineering layer provides real data and operational state, and the Agentic AI layer intelligently interacts with that data and operational knowledge.

The final system should be strong enough to discuss in Data Engineering / AI Engineering interviews and demonstrate real engineering concepts rather than toy functionality.

2. Current Architecture

The high-level architecture is:





DATA ENGINEERING

Olist Dataset | v Batch Simulation | v PySpark | v Airflow | v MySQL | +--> Analytics Tables | +--> Pipeline Metadata | +--> Watermarks

The Agentic AI layer sits ABOVE the Data Engineering layer.

Do not put LLM logic inside the core data pipeline unless there is a strong architectural reason.

3. Project Location

Main project:

D:\agentic-ai-adk

Current structure:

D:\agentic-ai-adk

```
| ├── .env/ | ├── data/ | ├── raw/ | | ├── olist/ | ├── batches/ | ├── processed/ | | ├── spark_pipeline/ | ├── init.py | ├── config.py | ├── schemas.py | ├── extract.py | | ├── transform.py | ├── validate.py | ├── load.py | ├── pipeline.py | | ├── batch_generator.py | ├── airflow/ | ├── dags/ | ├── olist_incremental_pipeline.py | | ├── agents/ | ├── business_query_agent/ | ├── pipeline_monitor_agent/ | ├── rag_agent/ | | ├── orchestrator/ | ├── api_server/ | ├── mcp_server/ | ├── .env | ├── requirements.txt
```

The exact contents may evolve, but avoid unnecessary restructuring.

4. Existing Environment

Windows:

- Project: D:\agentic-ai-adk
- Python: 3.14.7
- Google ADK: 2.8.0
- PySpark: 4.0.1
- Java 17
- MySQL Server: 8.0.42

- MySQL port: 3306

MySQL database:

agent_adk_db

MySQL user:

agent_adk_user

Environment variables in .env:

DB_HOST=localhost DB_PORT=3306 DB_USER=agent_adk_user DB_PASSWORD=<existing password>
DB_NAME=agent_adk_db

Never ask me to expose or paste my database password.

There is also an existing Google ADK / API / MCP system that is already working. Do NOT break or unnecessarily redesign it.

5. Why WSL Is Being Used

Apache Airflow is running inside WSL2 because native Windows Airflow is not supported.

WSL:

- Ubuntu 22.04
- Python 3.10.6
- Java 17
- Airflow 3.3.1
- Dedicated Airflow virtual environment:

~/airflow/venv

Airflow HOME:

~/airflow

The Windows project is accessible from WSL:

/mnt/d/agentic-ai-adk

Airflow DAG source is actually maintained in the Windows project:

/mnt/d/agentic-ai-adk/airflow/dags/olist_incremental_pipeline.py

WSL Airflow DAG location uses a symlink:

~/airflow/dags/olist_incremental_pipeline.py

which points to the project DAG.

Do not unnecessarily move the project or create another copy.

6. Current Airflow Setup

Airflow 3.3.1 is working.

Components currently used:

- DAG Processor
- Scheduler
- API Server

Airflow UI:

http://127.0.0.1:8080

Current DAG:

olist_incremental_pipeline

Current DAG flow:

find_next_batch | v run_pipeline | v verify_batch

The DAG successfully completed end-to-end.

The successful run proved:

Airflow → find_next_batch → PySpark incremental pipeline → Windows MySQL → pipeline_batches
SUCCESS → verify_batch → SUCCESS

The Airflow UI can now be used to manually trigger the DAG.

7. Olist Data

The project uses the Brazilian Olist e-commerce dataset.

Raw files are under:

data/raw/olist/

Main datasets:

- customers
- orders
- order_items
- payments
- products
- sellers
- product_category_translation

Approximate dataset sizes:

- customers ~99k
- orders ~99k
- order_items ~112k
- payments ~104k
- products ~33k
- sellers ~3k
- categories 71

Reviews and geolocation are currently not part of the core pipeline.

Olist is historical data, not live data.

To simulate incremental ingestion, the historical orders are divided chronologically into monthly batches.

There are currently 25 generated monthly batches.

Example:

batch_201609 batch_201610 batch_201612 ...

There is no batch_201611 because the source data does not contain that month.

8. Batch Architecture

batch_generator.py is a preparation utility.

It generates:

data/batches/batch_YYYYMM/

with:

orders/data.csv order_items/data.csv order_payments/data.csv

The batch generator has already successfully generated all batches.

It should NOT be treated as something Airflow runs every day.

Instead:

historical Olist data → one-time batch preparation → monthly batch folders → Airflow incrementally processes pending batches

9. MySQL Schema

Core/source tables:

1. customers
2. orders
3. order_items
4. products
5. sellers
6. order_payments
7. product_category_translation

Pipeline metadata:

1. pipeline_batches
2. pipeline_watermarks

Analytics:

1. monthly_sales_summary
2. category_performance
3. seller_performance

pipeline_batches tracks:

- batch_id
- source_name
- batch_date
- min_event_date
- max_event_date
- records_received
- records_processed
- records_failed
- status
- started_at
- completed_at
- error_message

pipeline_watermarks tracks:

- pipeline_name
- source_name
- watermark_column

- last_processed_value
 - updated_at
-

10. Current Incremental Logic

The Airflow DAG:

1. Finds all batch directories.
2. Reads successful batch IDs from pipeline_batches.
3. Determines the earliest pending batch.
4. Passes that batch ID to the PySpark pipeline.
5. PySpark reads the batch.
6. Data is transformed and validated.
7. Data is loaded into MySQL.
8. Analytics are refreshed.
9. pipeline_batches is marked SUCCESS.
10. Watermark is updated.
11. Airflow verifies the batch.

Important:

Airflow orchestrates.

PySpark processes and loads.

MySQL stores.

The LLM does NOT process 100k raw rows.

11. Phase Status

Phase 1 — Data Engineering Foundation

STATUS: COMPLETE

Completed:

- Olist ingestion
- PySpark
- schemas
- transformations
- validation
- MySQL loading
- analytics
- batch simulation
- incremental pipeline

- metadata
- watermarks
- Airflow integration
- WSL setup
- Airflow UI
- end-to-end execution

Phase 2 — Production Hardening

STATUS: DEFERRED INTENTIONALLY

Do NOT make this the immediate focus unless required by Phase 3.

Things we will revisit later:

1. Automatic Airflow scheduling
2. No-pending-batch handling
3. Retries/error handling
4. Stronger data-quality validation
5. Pipeline monitoring/logging

The current pipeline is working, so we deliberately decided not to modify it unnecessarily right now.

Phase 3 — Agentic Intelligence

STATUS: NEXT

This is where we are starting now.

First target:

Business Query Agent

12. Phase 3 — Business Query Agent Goal

Build an agent that allows a user to ask natural-language questions about the Olist business data.

Examples:

"What was the total revenue in 2017?"

"Which product category generated the most revenue?"

"What are the top 10 sellers by revenue?"

"What was the average order value in 2018?"

"Which month had the highest sales?"

The architecture should be:

User ↓ Business Query Agent ↓ Understand intent ↓ Generate / construct appropriate SQL ↓ Execute SQL against MySQL ↓ Return small aggregated result ↓ LLM explains result ↓ User

IMPORTANT:

The LLM should NOT receive the entire database or raw 100k-row datasets.

SQL/database operations should perform filtering, aggregation, joins, etc.

Only the necessary result should be passed to the LLM.

13. Business Query Agent Design Principles

We want this to be a REAL agent, not simply:

LLM → SQL string → database

The agent should eventually be able to:

- understand the question
- identify relevant tables
- identify relevant metrics
- generate safe SQL
- execute SQL
- inspect the result
- explain the result
- handle ambiguity
- potentially retry/correct a failed SQL query
- avoid destructive SQL
- provide useful business-level answers

We need to decide the exact tool architecture before implementing.

Potential tool:

`execute_read_only_sql()`

The tool should enforce read-only behavior.

We should consider:

- SELECT-only SQL
- query limits
- timeout

- table allowlisting
- protection against INSERT/UPDATE/DELETE/DROP/ALTER
- clear database errors
- result-size limits

Do not over-engineer the first version, but build it in a way that can evolve.

14. Agent Framework

The project uses Google ADK.

Use the existing ADK setup and conventions rather than introducing another agent framework unless there is a strong reason.

The project may eventually use:

Google ADK

+ MCP tools

+ MySQL

+ RAG

+ Orchestrator

But Phase 3 should focus on getting the Business Query Agent working cleanly first.

15. Future Agent Roadmap

After Business Query Agent:

Pipeline Monitor Agent

Questions such as:

"Did the latest pipeline run successfully?"

"Which batch was processed last?"

"How many records were processed?"

"Did any batch fail?"

"What is the current watermark?"

It will use:

pipeline_batches pipeline_watermarks

RAG Agent

Knowledge sources may include:

- data dictionary
- schema documentation
- pipeline documentation
- business definitions
- runbooks
- incident documentation
- troubleshooting guides

Root Orchestrator

Eventually:

User ↓ Root Orchestrator ↓ decides which agent(s) are needed ↓ Business Query Agent Pipeline Monitor Agent RAG Agent ↓ combine results ↓ final answer

16. MCP

There is already an existing MCP server / employee-related system.

Do NOT break it.

MCP may later be used to expose useful tools/capabilities to agents.

Do not force MCP into every component just for the sake of using MCP.

Use it where it provides a meaningful architectural benefit.

17. Important Engineering Principles

Please follow these throughout the project:

1. Do not unnecessarily redesign working components.
2. Prefer production-like architecture over toy demos.
3. Keep components modular.
4. Keep Data Engineering and Agentic AI layers separated.
5. Database performs data-intensive operations.
6. LLM receives only relevant/small results.
7. Use explicit schemas and validation.

8. Make failures observable.
 9. Prefer deterministic operations where possible.
 10. Don't add technologies just to increase the technology list.
 11. Explain WHY a design is being chosen.
 12. If I propose something technically inferior, tell me directly.
 13. If a change could break an existing working component, warn me before implementing it.
 14. Give copy-paste-ready commands when setup/debugging is required.
 15. Keep instructions concise and implementation-focused.
 16. Don't make me repeat project context unnecessarily.
 17. When debugging, ask for the exact command/output rather than guessing.
-

18. Current Immediate Task

We have finished the Data Engineering foundation and have intentionally deferred Phase 2.

The pipeline is currently working.

Therefore, start with:

PHASE 3 — BUSINESS QUERY AGENT

Before writing substantial code:

1. Review the existing architecture.
2. Decide the Business Query Agent architecture.
3. Identify exactly which MySQL tables it needs.
4. Decide the tool interface.
5. Decide how SQL generation/execution should work.
6. Decide how to enforce read-only queries.
7. Decide how Google ADK should structure the agent and tools.
8. Identify whether the existing API/MCP infrastructure can be reused without breaking it.
9. Then implement incrementally and test each component.

Do not jump directly into a large code dump.

First establish the Phase 3 design, then implement it step by step.

The objective is to build this system together incrementally and keep the architecture coherent all the way to the final portfolio/GitHub version.